Alternative Language Support in Smalltalk Image

Adrien Hopkins[1], Dave Mason[1]

[1]*Toronto Metropolitan University, Toronto, Canada*

Abstract

Smalltalk[1] systems have three primary components: syntax, semantics, and the image. Implementing the semantics in a runtime or virtual machine is a major engineering task and a requirement for the other two. However, the syntax and the image are, in some sense less fundamental. GNU Smalltalk[2, 3] demonstrates that the syntax and semantics can be divorced from a requirement for an image and remain useful. This paper considers divorcing the other end - the Smalltalk syntax.

1. Introduction

While Smalltalk syntax is beloved by many (including the authors of this paper), there are a variety of reasons why it might be useful to support alternatives:

Unfamiliarity. Most developers have no experience with Smalltalk and often think it “looks weird" and therefore miss out on the highly productive Smalltalk environment.

Community size. While “many" like Smalltalk syntax, “many" isn’t as many as we might like, and managers are often willing to ignore Smalltalk’s benefits because of unfamiliarity or fear of being unable to find developers.

LLM Support. While LLMs are somewhat language agnostic, the available training data for e.g. Python is vastly larger than that for Smalltalk.

Library support. Many popular and powerful libraries are implemented in Python, such as Pandas[4] for data management, Matplotlib[5] for data visualization, and TensorFlow[6] for machine learning. Translation is one way these libraries could be used in Smalltalk, in addition to foreign function interfaces.

The Zag Smalltalk system[7, 8] does not store text for methods, but rather annotated abstract syntax trees (ASTs). This means that a language that is semantically close enough to Smalltalk could be translated from the source language to an AST. Furthermore, an AST could be displayed in alternative source languages. In principle, this means that each developer could develop and debug in the language of their choice. This is a more integrated approach than that in [9]. As opposed to the interpreter-generation approach of [10], this is an attempt to support multiple languages in a single runtime.

This is not a completely novel idea, since MagLev[11] explored the implementation of Ruby[12] on the Gemstone/S[13] Smalltalk VM.

In the image, the user will be able to choose a language in which they want to interact. If they choose Smalltalk, they will see the conventional representation of the method, whether in the browser or a debugger. If they choose another language, such as Python, they will see the same series of windows, but the actual code would show the Python equivalent. When looking at a method, if the language the user has selected is the same as the language it was entered in, they should see exactly what was entered. If the language is different there may be some artifacts, although the long-term goal is to minimize that.

IWST 2026: International Workshop on Smalltalk Technologies, July 7th to 10th, 2026, Plovdiv, Bulgaria

2. Python and Smalltalk

Python[14, 15] is by far the most popular dynamically-typed language at the time of writing, with many useful libraries available for various applications. Therefore, it is a natural candidate for support in our system.

Python has major similarities with Smalltalk, which makes translation easier than with most other language pairs:

- Python and Smalltalk are both dynamically typed. If one of the languages were statically typed, then translation would involve guessing the types of every variable, or casting types in order to be able to execute methods.
- Every value in both Python and Smalltalk is an object that exists within the language's class hierarchy.

2.1. From Python to Smalltalk

The actual implementation is relatively straight-forward. Currently we are using tree-sitter[16] to parse the Python source into a Python AST. Then we use a visitor pattern to translate the tree-sitter AST to the Zag (Smalltalk) AST. We may switch to a bespoke parser in future, but tree-sitter makes it easy to quickly build a prototype.

The tree-sitter AST only describes the structure of the program; it does not include details such as the names of variables or the specific operator that a node represents. The only instance variables that tree-sitter nodes have are:

- the type of the node, such as "if statement" or "binary operator", and
- the start and end positions of this node in the source code.

In order to parse the tree-sitter AST, we created our own node objects that contain both the tree-sitter node and a copy of the source code. Details like variable names and specific operators can then be easily obtained by reading the node's part of the source code, sometimes involving basic parsing.

Although Python is similar to Smalltalk, there are many interesting challenges when translating from Python to Smalltalk:

- Python uses truthy/falsey boolean values, so whenever a boolean test is being done, an special message called `toPythonBoolean` must be sent before messages such as `ifTrue:ifFalse:`. This method is used to minimize the number of new messages that had to be added, given that we intend to translated multiple languages, several of which have their own truthiness semantics. When looking at the method through a Smalltalk lens, this will be visible, but it also means this method is available to Smalltalk code.
- Boolean operators always return one of their arguments, meaning that "1 and 1" is not the same as "True and True", even though 1 is truthy. This is useful behaviour which should be kept in translated code, but is not possible to replicate using `toPythonBoolean`. To solve this, boolean operators are translated into the special messages `pythonAnd:` and `pythonOr:`.
- Python conditionals and loops will be translated to the appropriate message sends. When looking at the method through a Python lens, we must recognize the pattern and turn it back into an if or while statement.
  A custom switch statement was added as an extension, so that long if/elif chains can be translated without the excessive nesting that would be required if using `ifTrue:ifFalse:`. It is only used if the Python code has an `elif` statement, otherwise either `ifTrue:` or `ifTrue:ifFalse:` is used.
- Python has `break` and `continue` statements, which can be used to end a loop iteration early. Smalltalk has no such functionality. `break` needs a special version of non-local returns to implement correctly. Similarly, `continue` uses Zag's tail-send operation to restart the appropriate block.

- Python uses multiple inheritance, so this will be translated to the corresponding trait. There will have to be a way to display a trait if it implements a more exotic inheritance structure.
- Primitive methods like `__add__` have fixed arity so they can be translated directly to use the Smalltalk + methods.
- Regular methods can be variadic and have an optional dictionary as the last parameter, so a new send mechanism must be supported.
- Python functions can be defined inside other functions, and returning returns from the innermost function. Smalltalk messages cannot be defined inside other messages. Smalltalk blocks can be defined inside messages or other blocks, but they do not have a return statement, instead returning the value of their last statement.
- Arrays are indexed from 0 instead of 1, and negative indices can be used to access elements near the end of an array (-1 = last, -2 = second-last, etc.).
  This was solved by adding custom messages such as `zwAt:`, which is like `at:` but with Python's zero-indexed index-wrapping behaviour. Then, the array index operation can be translated to `zwAt:` instead of `at:`, and similar things can be done with other index-requiring operations.

Though the system does not yet support translating arbitrary library calls, a few specific library calls were given manual translation rules in order to make it possible to test the system with fully functional programs:

- Python's `print(x)` method is translated into the code `Transcript show: x; cr`.
- Python's `input(x)` method is translated into the code `UIManager default request: x`.
- The datatype conversions `int(x)` and `float(x)` are translated into `x asInteger` and `x asFloat` respectively.

These library calls were added as they became necessary to translate simple programs. Most library calls will not be translated via manually implemented rules. They will instead use the system discussed in section 2.3.

2.2. From Smalltalk to Python

We are only in the beginning phases of creating the translator from Smalltalk to Python, unlike the other direction.

We will likely not be able to use tree-sitter to convert from Smalltalk to Python. This is because, as mentioned in the previous section, the tree-sitter AST does not contain a program's full description, only its structure. Translating a tree-sitter tree to Python source code is likely impossible without extra information.

Instead, we are planning to create a Python AST, similar to the Zag AST but for Python instead of Smalltalk. The specific types of nodes will likely be similar to the tree-sitter structure, but elements will have instance variables describing all relevant details such as variable names and specific operators.

Then, programs can be translated from the Zag AST to this Python AST, then from this Python AST to Python source code. The first step will look very similar to the conversion from Python to Smalltalk, but with the conversion rules performed in reverse. The second step will also involve the use of a visitor pattern similar to the other tree conversions.

2.3. Future Challenge: Library Methods

One major challenge we expect in the future is converting library functions between Python and Pharo. There are a very large number of these functions in both languages, meaning that creating manual translation rules for every library function is not possible in any reasonable amount of time.

We are considering training and using an LLM to translate these calls. This LLM would convert function call nodes in one language to subtrees in another, converting each call separately.

This approach is intended to combine the best aspects of traditional programming and LLMs: traditional programming ensures high accuracy and provides control over the output, but requires large amounts of effort to implement, while LLMs are not always accurate or predictable, but can be implemented within a reasonable amount of time. Given that programs' basic structure is converted using manual rules, we expect most translated programs to at least compile regardless of the accuracy of the LLM system.

This is merely a plan for the future and has not yet been implemented, so everything in this section is subject to change.

3. Zag and Opal

While the target for this work is the Zag Smalltalk system, it currently doesn't support a GUI, so we have added the capability to generate Opal[17] ASTs from any Zag AST. In this section we describe how that works.

The implementation is fairly similar to the implementation of the Python-to-Zag converter described in the previous section. Given that the Zag AST and Opal AST are both Smalltalk ASTs, they are incredibly similar. Hence, this system is much simpler than the Python-to-Zag converter; all the conversion methods are one or two lines of code excluding variable declarations.

Every element of the Zag AST is a subclass of the `AST` class. They all share an `asPharoAST` message. The basic leaf elements, such as `ASLiteral`, simply return the associated Opal AST classes, while more complicated ones that contain other AST parts as variables convert by recursively sending `asPharoAST`.

Once you have an Opal AST, you can execute the generated AST as a Smalltalk program. The Opal compiler is able to accept an AST via the code `OpalCompiler new ast: myAST`, and it can execute this code by sending this compiler object the `evaluate` message.

4. Evaluation

4.1. Present Evaluation Procedures

Currently, not too much testing and evaluation has been done for the system, as it is still in the construction phase. However, as of the writing of this paper, we have 51 automatic tests for the tree-sitter to Zag converter, including 2 tests that test converting small, functional programs from Python source code to the Zag AST. They are:

- a simple, three-line Fahrenheit to Celsius converter, and
- a very barebones guessing game.

All automatic tests are run before making a commit to this project's git repository, and changes are only committed if all automatic tests pass.

We also manually tested a full conversion of the Fahrenheit to Celsius converter as per the method described in section 3, and the program executed sucessfully.

4.2. Future Evaluation Procedures

We will continue to add automatic tests, including more full programs, as the system is further developed. In addition, once the system is more complete, we will perform more extensive testing. One idea for testing the system is generating programs to solve the same problem in multiple ways, and ranking the methods in terms of program accuracy. The methods we will test include:

- Python programs generated by LLMs or written manually by a skilled programmer,
- Smalltalk programs generated by the same source as the Python programs, and
- programs generated in Python then translated to Smalltalk using our transpilation system.

We expect the non-translated Python programs to perform the best; they are included as the control for this experiment. Our system will be considered to work very well if the translated Smalltalk programs are more accurate than LLM-generated Smalltalk programs.

We have not decided how to determine the accuracy of the generated programs. Some metrics we are considering are:

- the proportion of generated programs that compile successfully,
- the proportion of test cases passed by generated programs,
- the amount of time it takes for a skilled programmer to fix programs such that they pass all test cases, and
- the similarity between generated and fixed programs.

As our system is not yet complete, the specific evaluation procedures are subject to change.

5. Conclusions

Programming language translation is an interesting and useful field with several applications, including aiding LLM generation and allowing programs of different languages to be combined together. As part of the Zag compiler, we are in the early stages of implementing a system that allows the same code to be represented as either Smalltalk or Python, and freely translated between the two. We have already succesfully translated basic programs from Python to Smalltalk, and once the system is more developed, the system will be able to translate arbitrary programs between Python and Smalltalk.

Much work remains in the future: completing Python-to-Smalltalk translation for all language features, translating Smalltalk to Python, translating library calls, and more. Many more challenges are expected as we complete the system and support more and more programs.

References

[1] A. Goldberg, D. Robson, Smalltalk-80: The Language and Its Implementation, Addison-Wesley, 1983. The 'Blue Book' defines the core language GNU Smalltalk implements.

[2] GNU Project, GNU Smalltalk User's Guide, Free Software Foundation, 2024. URL: https://www.gnu.org/software/smalltalk/manual/gst.html, version 3.2.91.

[3] S. Ducasse, Computer Programming with GNU Smalltalk, Square Bracket Associates / Free Online Edition, 2009. URL: http://stephane.ducasse.free.fr/FreeBooks/Gnu/ProgrammingUsingGnuSmalltalk.pdf.

[4] Pandas authors, Pandas, 2026. URL: https://pandas.pydata.org/.

[5] Matplotlib authors, Matplotlib: Visualization with python, 2026. URL: https://matplotlib.org/.

[6] TensorFlow authors, Tensorflow, 2026. URL: https://github.com/tensorflow/tensorflow.

[7] D. Mason, Design principles for a high-performance Smalltalk, in: International Workshop on Smalltalk Technologies (IWST 2022), 2022. URL: https://sarg.torontomu.ca/dmason/UKSt2022-11-30.pdf.

[8] D. Mason, Zag smalltalk, https://github.com/Zag-Research/Zag-Smalltalk, 2025. Accessed: 2025-04-08.

[9] F. Niephaus, T. Felgentreff, T. Pape, R. Hirschfeld, Squeak makes a good python debugger: Bringing other programming languages into smalltalk's tools, in: Companion Proceedings of the 1st International Conference on the Art, Science, and Engineering of Programming, Programming '17, Association for Computing Machinery, New York, NY, USA, 2017. URL: https://doi.org/10.1145/3079368.3079402. doi:10.1145/3079368.3079402.

[10] C. F. Bolz, A. Kuhn, A. Lienhard, N. D. Matsakis, O. Nierstrasz, L. Renggli, A. Rigo, T. Verwaest, Back to the future in one week — implementing a smalltalk vm in pypy, in: R. Hirschfeld, K. Rose (Eds.), Self-Sustaining Systems, Springer Berlin Heidelberg, Berlin, Heidelberg, 2008, pp. 123–139.

[11] M. Springer, Inter-language collaboration in an object-oriented virtual machine, 2016. URL: https://arxiv.org/abs/1606.03644. arXiv:1606.03644.
[12] D. Flanagan, Y. Matsumoto, The Ruby Programming Language, O'Reilly Media, Sebastopol, CA, 2008.
[13] GemTalk Systems, GemStone/S 64 Bit Programmer's Guide, GemTalk Systems, 2024. URL: https://downloads.gemtalksystems.com/docs/GemStone64/3.7.x/GS64-ProgGuide-3.7/1-IntroToGemStone.htm, version 3.7.x.
[14] Python Software Foundation, Python Language Reference, version 3.x, Python Software Foundation, 2026. URL: https://www.python.org/.
[15] G. van Rossum, Python tutorial, Technical Report CS-R9526, Centrum voor Wiskunde en Informatica (CWI), Amsterdam, 1995.
[16] Tree-sitter authors, Tree-sitter, 2026. URL: https://tree-sitter.github.io/.
[17] C. Béra, M. Denker, Towards a flexible pharo compiler, in: International Workshop on Smalltalk Technologies (IWST 2013), Annecy, France, 2013. URL: https://inria.hal.science/hal-00862411v1/document.